

VULNERABILITY ASSESSMENT REPORT

Read-Only,PassiveSecurity Review

Target Website: http://testphp.vulnweb.com/ [replace with your target]
Prepared by: THOTA JANAKI RAM KRISHNAMA NAIDU
Date: 2 Aug 2026
Prepared for: Future Interns – Cyber Security Task 1 (2026)

1. Executive Summary

This report presents the results of a passive, read-only vulnerability assessment performed against [target website]. The objective was to identify common, non-intrusive security weaknesses — such as missing security headers, outdated components, and information disclosure — and to translate them into clear, business-friendly risk guidance.

No exploitation, login bypass, brute-force, or denial-of-service activity was performed at any point. All testing was limited to publicly accessible pages using passive scanning and configuration review.

Overall summary: [X] High, [X] Medium, and [X] Low risk findings were identified. [One-sentence takeaway for a business owner, e.g. "No critical data exposure was found, but several quick configuration fixes are recommended."]

2. Scope & Ethics

In Scope

- Public-facing pages of [target website] only
- Passive scanning (OWASP ZAP passive mode)
- HTTP response header analysis
- Basic port / service exposure review (Nmap, non-intrusive)
- Client-side configuration review (Browser DevTools: cookies, headers, mixed content)

Out of Scope / Not Performed

- Login bypass or authentication attacks
 - Active exploitation of any kind
 - Brute-force attacks
 - Denial-of-Service (DoS) testing
- Any action that could disrupt, damage, or degrade the target site

This assessment follows an ethical, consulting-style approach: the goal is to identify and explain risk, not to demonstrate exploitation.

3. Methodology & Tools

Tool	Purpose
Nmap	Basic port and exposed-service enumeration (non-intrusive scan)
OWASP ZAP (Passive Scan)	Identifies vulnerabilities from traffic already generated by normal browsing, without sending attack payloads
Browser DevTools	Inspects response headers, cookies (Secure/HttpOnly flags), and mixed-content warnings
curl -I	Quick command-line capture of raw HTTP response headers for evidence

4. Summary of Findings

The table below lists all findings identified during the assessment, ordered by risk level.

#	Vulnerability	Risk Level	Affected Area
1	Missing security response headers (CSP, X-Frame-Options, HSTS)	Medium	All pages
2	Outdated / unmaintained server software disclosed in headers	Medium	Server: header
3	Verbose error messages reveal internal file paths or SQL syntax	High	Search / listing pages
4	Directory listing enabled on non-essential paths	Low	/pictures, /images
5	Session cookies missing Secure / HttpOnly flags	Medium	Login / session cookies
...	[Add your own findings here]		

5. Detailed Findings

Each finding below follows the same structure: what the issue is, why it matters to the business, supporting evidence, and a practical fix.

5.1. Missing Security Response Headers [Medium Risk]

What is the issue?

The website does not return recommended security headers such as Content-Security-Policy (CSP), X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security (HSTS). These headers were checked using curl -I and confirmed in Browser DevTools' Network tab.

Why does it matter?

Without these headers, browsers are not told how to defend the site against common attacks like clickjacking (embedding the site in a hidden frame) or content-type sniffing. This does not mean the site is actively being attacked, but it removes a free layer of built-in browser protection.

Evidence

[Paste curl -I output or a DevTools screenshot showing the response headers here.]

Recommended Fix

Add the missing headers at the web server or CDN level. A reasonable starting baseline: Content-Security-Policy: default-src 'self'; X-Frame-Options: DENY; X-Content-Type-Options: nosniff; Strict-Transport-Security: max-age=31536000. Most hosting platforms and reverse proxies (Nginx, Apache, Cloudflare) support this with a few lines of configuration.

5.2. Outdated/Version-Disclosing Server Banner [Medium Risk]

What is the issue?

The Server response header discloses the exact web server software and version (e.g., 'nginx/1.19.0'). This was identified via curl -I and Nmap service detection.

Why does it matter?

Publishing exact software versions makes it trivial for an attacker to look up known vulnerabilities for that specific version without any extra scanning effort. It is a low-cost fix with real reduction in reconnaissance value for an attacker.

Evidence

[Paste the raw Server: header value here.]

Recommended Fix

Disable or minimize the Server header (e.g., 'server_tokens off;' in Nginx, or an equivalent directive in Apache/IIS), and keep server software patched to the latest stable release on a regular schedule.

5.3. Verbose Error Messages / Information Disclosure [High Risk]

What is the issue?

Certain pages returned detailed error output (e.g., database or application error text) when unexpected input was passed through normal browsing/navigation, rather than a generic error page.

Why does it matter?

Detailed errors can reveal internal file paths, database structure, or the underlying technology stack. This significantly shortens the amount of work an attacker needs to do to plan a targeted attack, and in some cases can leak data directly.

Evidence

[Screenshot or copy of the exact error text observed, with the URL/path that triggered it.]

Recommended Fix

Configure the application/server to show generic, user-friendly error pages in production, and log detailed errors only on the server side where the public cannot see them.

5.4. Directory Listing Enabled [Low Risk]

What is the issue?

Browsing directly to certain folder paths (e.g., an images or assets directory) returned a raw file listing instead of a 404 or a proper page.

Why does it matter?

This isn't dangerous by itself, but it can expose file names, naming conventions, or files that were never meant to be public, which is useful reconnaissance information for an attacker.

Evidence

[Screenshot of the directory listing page.]

Recommended Fix

Disable directory listing at the web server level (e.g., 'autoindex off;' in Nginx) and add an index file to each public directory.

5.5. Cookies Missing Secure / HttpOnly Flags [Medium Risk]

What is the issue?

Session or authentication-related cookies were observed (via DevTools > Application > Cookies) without the Secure and HttpOnly attributes set.

Why does it matter?

Without HttpOnly, a cookie is readable by JavaScript, increasing exposure if a cross-site scripting issue exists elsewhere on the site. Without Secure, the cookie could be sent over an unencrypted connection.

Evidence

[Screenshot of the Cookies panel in DevTools showing the flags.]

Recommended Fix

Set Secure and HttpOnly (and SameSite=Lax or Strict where appropriate) on all session and authentication cookies.

6. Risk Classification Summary

Risk levels in this report follow a simple, business-friendly scale:

- High — Could lead to meaningful data exposure or loss of control if left unaddressed; fix urgently.
- Medium — Weakens the site's defenses and should be scheduled for near-term remediation.
- Low — Minor hygiene issue; fix when convenient, low urgency.

7. Recommendations & Next Steps

- Prioritize fixes in order: High risk items first, then Medium, then Low.
- Re-run this same passive review after fixes are applied to confirm they resolved the issue.
- Consider a scheduled quarterly review, since server software and headers can drift out of date over time.
- For anything beyond passive review (e.g., authenticated or active testing), engage a qualified penetration tester under a signed authorization / scope agreement.

— End of Report —